```
class PictureFile < ActiveRecord::Base
  after_destroy PictureFileCallbacks.new
end
```

Note that we needed to instantiate a new `PictureFileCallbacks` object, since we declared our callback as an instance method. This is particularly useful if the callbacks make use of the state of the instantiated object. Often, however, it will make more sense to declare the callbacks as class methods:

```
class PictureFileCallbacks
  def self.after_destroy(picture_file)
    if File.exist?(picture_file.filepath)
      File.delete(picture_file.filepath)
    end
  end
end
```

If the callback method is declared this way, it won't be necessary to instantiate a `PictureFileCallbacks` object.

```
class PictureFile < ActiveRecord::Base
  after_destroy PictureFileCallbacks
end
```

You can declare as many callbacks as you want inside your callback classes.

10 Transaction Callbacks

There are two additional callbacks that are triggered by the completion of a database transaction: `after_commit` and `after_rollback`. These callbacks are very similar to the `after_save` callback except that they don't execute until after database changes have either been committed or rolled back. They are most useful when your active record models need to interact with external systems which are not part of the database transaction.

Consider, for example, the previous example where the `PictureFile` model needs to delete a file after the corresponding record is destroyed. If anything raises an exception after the `after_destroy`callback is called and the transaction rolls back, the file will have been deleted and the model will be left in an inconsistent state. For example, suppose that `picture_file_2` in the code below is not valid and the `save!` method raises an error.

```
PictureFile.transaction do
  picture_file_1.destroy
  picture_file_2.save!
end
```

By using the `after_commit` callback we can account for this case.

```
class PictureFile < ActiveRecord::Base
  after_commit :delete_picture_file_from_disk, on: [:destroy]

  def delete_picture_file_from_disk
    if File.exist?(filepath)
      File.delete(filepath)
    end
  end
end
```

the `:on` option specifies when a callback will be fired. If you don't supply the `:on` option the callback will fire for every action.

The `after_commit` and `after_rollback` callbacks are guaranteed to be called for all models created, updated, or destroyed within a transaction block. If any exceptions are raised within one of these callbacks, they will be ignored so that they don't interfere with the other callbacks. As such, if your callback code could raise an exception, you'll need to rescue it and handle it appropriately within the callback.